

Samana QMS: architecture review, and what was changed

22 September 2026 · Second review, read as an enterprise architect · Code at commit 78705c2 · Written for management, engineering, and whoever reviews this next

Verdict: production-ready, and now defensible line by line. This is a stricter pass than the external review of 21 September. It read every server module, the data hooks, the workspace shell, the schema and the tests, and judged them against enterprise practice rather than against the earlier review's list. It found 15 things worth changing in the application and 2 in the Salesforce client. All 17 are done, each with a test or a measured proof, and the decisions kept on purpose are defended below with their cost of reversal.

For the next reviewer: every claim in this document is reproducible with the commands in section 6, on any PostgreSQL 14 or newer server, with no external secret.

17 findings, all closed	175 automated tests across 16 files, in two runtimes	0 inline scripts allowed by the browser policy
4 files that may read the environment, enforced by a test	91 dead stylesheet blocks removed with a proof	82 overall score, from 77 this morning and 69 on 21 September

1. Scorecard

Scores are out of 100 per dimension, judged before and after the changes in this document. The method is the same as the first review so the movement is comparable; the number is a summary, the rows that follow are the substance.

Dimension	Before	After	What moved it
Database design and integrity	88	90	Indexes for every query that runs on a timer; one atomic row per rate limit.
Documentation	88	88	Unchanged in kind; every rule below is now in the requirements, the runbook and the example environment file.
Backend architecture	80	86	One validated configuration boundary; typed error identity; routing and delivery on separate timers.
Operations and deployment	78	84	Boot-time validation that refuses a half-configured deploy; graceful shutdown; coverage gate in CI.
Security	78	86	Nonce-based script policy with no inline allowance; atomic limiter; the cookie's Secure flag must match the origin's scheme.
API design	76	84	A stable code on every error; 500 only for unexpected faults; no status decided by words inside a message.
Testing	74	82	A browser test project for the screens; configuration and policy suites; the lifecycle scenario isolated; coverage measured and gated.
Performance and scale	72	80	The three sequential scans on hot paths are gone. Substring search and offset paging remain, and are noted.
Maintainability	72	80	Role groups defined once for both sides; the silent duplicate rules deleted; timers out of the shell.
Frontend architecture	66	76	Framework router; refs written in effects; role predicates shared; component tests. Three stylesheets remain.
Observability	64	68	Configuration and shutdown events logged. Metrics and tracing remain future work, and are not needed for one centre.
Overall	77	82	

2. Findings, and what was done about each

Severity is as judged before the change. "Proof" names the test that now fails if the fix regresses, or the measurement taken.

#	Finding	Severity	What changed	Proof
1	No validated configuration boundary. Eighteen files read the environment directly; nothing checked it at start, so a missing QR secret first appeared as a 503 in front of a customer and a misspelled cookie flag silently changed behaviour.	High	One module reads and validates every setting once and freezes it. The web server and the scheduler call it before serving anything; a bad deployment stops at boot with a message naming the variable. Empty strings mean unset, as hosting dashboards express it. Contradictions are refused: SMS on without a gateway, write-back on without a connection, a Secure cookie on an <code>http://</code> origin (browsers would drop it and nobody could sign in), an insecure cookie on <code>https://</code> . A test pins that only four files may touch the environment, each for a stated reason.	<code>tests/config.test.ts</code> (16 cases); <code>tests/boundary.test.ts</code> "reads the environment in one place"
2	Content Security Policy allowed inline scripts. For a system holding Emirates ID and passport numbers, <code>'unsafe-inline'</code> removed most of the protection the policy exists to give.	High	A nonce is minted per request; the policy allows scripts only by that nonce with <code>'strict-dynamic'</code> , and the renderer stamps it on its bootstrap script. Every page now renders per request, because a page prerendered at build time would carry a stale nonce. Inline styles remain allowed: a style cannot execute code, and the design system sets them.	Built server, real browser: 15 script tags, 15 carrying the nonce, 0 without; login form hydrated; 0 policy violations. <code>tests/proxy.test.ts</code> (6 cases)
3	Three hot queries had no index. Unread notifications on every queue refresh, the outbox claim every 15 seconds, the lookup reuse check on every check-in: all planned as sequential scans.	High	Migration 016 adds a partial index on unread notifications by user, an index on outbox readiness, one on lookups by identifier and time, and one on audit events by action. Applied to both local databases and, on deploy, to production.	<code>db/016_hot_path_indexes_and_limits.sql</code> ; schema snapshot test
4	Routing and delivery shared one loop. A slow Salesforce delayed presence expiry and five-minute re-routing by however long deliveries took.	Medium	The scheduler runs two independent, non-overlapping timers: routing every 15 seconds, delivery and retention on their own cadence. It also drains the pool on shutdown.	<code>scripts/worker-entry.ts</code> ; smoke test ticks
5	The rate limiter had a window race. Two simultaneous first attempts each saw no live window and opened their own, briefly doubling the budget.	Medium	One row per key; the whole decision is a single upsert with the window reset inside the statement. The migration changes the primary key and clears the transient counters.	<code>tests/security.test.ts</code> (4 limiter cases)
6	Unknown errors returned 503 , telling monitors and load balancers the service was down when one request failed.	Medium	Unexpected faults are 500 with a request id and a stable code; 503 is reserved for a dependency that is genuinely unavailable.	<code>tests/api.test.ts</code> "keeps provider errors distinct from no match"
7	Errors were mapped by substring. A database message containing <code>FORBIDDEN</code> , or a Salesforce message containing <code>(404)</code> , decided the HTTP status.	Medium	Error identity is a field. Database rules map by <code>SQLSTATE P0001</code> and the exact rule name; the Salesforce client carries the org's status on a typed error; the browser client exposes a typed error with status and code. Rewording a message can no longer change a status.	<code>lib/errors.ts</code> ; every existing API test passes on the new mapping
8	Role lists were duplicated in the browser , the defect the first review raised, surviving on the client.	Medium	Four role groups are defined once in the domain module. The route table declares access with them and the workspace decides what to show with them; there is no second list.	<code>lib/domain.ts</code> ; no role array remains in components
9	Navigation bypassed the framework router with a hand-rolled history listener.	Medium	The address is the view. Next.js keeps the two in step, including back and forward.	<code>components/qms/app.tsx</code>
10	Refs were mutated during render in both data hooks.	Medium	Mirrors are written in effects, so a render React discards leaves nothing behind.	<code>use-queue.ts</code> , <code>use-live.ts</code>
11	Dead business rules. Two TypeScript rules were tested and exported but called by nothing, because the real rules live in SQL.	Medium	Deleted, with their tests. The database functions are the one definition and 29 workflow tests exercise them.	<code>lib/domain.ts</code>
12	No graceful shutdown.	Low	On SIGTERM the web server closes the pool and the change listener; the scheduler finishes its tick and closes the pool.	<code>lib/lifecycle.ts</code> , <code>scripts/worker-entry.ts</code>
13	Scanning the check-in link on a TV signed the TV out.	Low	A display session is kept; the request is refused with a message to scan with a phone.	<code>lib/public-access.ts</code>
14	The application role may execute every function.	Low	Kept, with the reason in section 4.	—

#	Finding	Severity	What changed	Proof
15	No coverage metric, no component tests, order-dependent API tests.	Medium	A second test project runs component tests in a DOM for the check-in screen and the live queue table, driven the way a person uses them. Coverage is measured over the shipped code and gated by thresholds in CI. The ticket-lifecycle tests are a named scenario that issues its own ticket.	tests/ui/ (14 cases); vitest.config.ts ; npm run test:coverage
16	Salesforce health probe called the org twice every time Settings opened.	Low	Held for 60 seconds. Health a minute old is still health.	lib/salesforce.ts
17	A malformed response body tripped the circuit breaker.	Low	Only a failed connection or a 5xx opens the breaker; an unreadable body is reported as a bad response and the org is not paused.	lib/salesforce.ts

Also done in passing: the last non-null assertion in the codebase is gone, the delivery worker checks the ticket still exists before building a payload, and the three timers in the workspace shell (heartbeat, toast, QR link) moved into one hook so the shell reads as layout and state.

3. Proofs

Claim	Evidence	How it was obtained
The nonce policy does not break the application	Served HTML carried 15 script tags, all 15 with the request's nonce and none without; the policy contained no inline allowance and 'strict-dynamic' ; a headless browser loaded the page, executed the scripts and rendered the login form, which only client-side code produces; the browser reported zero policy violations.	Built standalone server on a spare port; headers and HTML fetched; Microsoft Edge headless with console logging, DOM dumped after load
A bad deployment cannot serve traffic	The first run with the new validation refused to start: it named both contradictory values. The rule was corrected (see section 4, "why not NODE_ENV") and the server then started, logging the loaded configuration.	Server log from the same run
Nothing else regressed	175 tests pass across 16 files in two runtimes; type check and lint clean; both builds succeed; the 30-step end-to-end smoke test passes on the built server and scheduler against the company PostgreSQL 18 server (health, alerts, sign-in, forced password change, presence, Salesforce sandbox probe, QR invite, guest session and lookup, ticket issue and its idempotent retry, public status, call, TV board, stale version refused, start, offline-mid-service refused, close with note, history, receipt, reports and CSV, audit, walk-in, live stream, paging validation, 404s, headers, sign-out).	npm run typecheck , npm run lint , npm run test:coverage , npm run build , npm run build:worker , smoke script
The stylesheet pruning changed no style	1,291 rule blocks parsed with their at-rule context. 91 were fully shadowed: a later block with the same selector in the same context set every property the earlier one set, and the earlier used no !important . At-rules and keyframes were excluded. After removal, the surviving 1,200 blocks were re-parsed and compared to the kept set: identical, in order, and nothing left shadowed. The application then built.	Pruning script, run twice: analyse, then apply and verify
The three hot queries were sequential scans	EXPLAIN on each showed Seq Scan before migration 016.	Query plans on the test database
The queue engine under load, after the indexes	Re-measured after migration 016 on the same laptop, now at 58% background CPU (OneDrive, the editor, mail): 200 simultaneous check-ins in 2.2 to 3.1 s, the routing sweep at 200 to 260 ms with 200 waiting and 20 agents online, 600 parallel agent actions at a median of 57 to 76 ms. To separate the indexes from the machine, the four indexes were dropped and recreated back to back with a run each way: 2.9 s / 213 ms / 64 ms without them, 2.2 s / 212 ms / 57 ms with them, 2.4 s / 200 ms / 57 ms without them again. The indexes change nothing measurable on a 30-ticket queue, as expected; they exist for the year-old table. The morning figures on a quiet machine (1.3 s, 96 ms, 23 ms) remain the reference, and the slower afternoon figures are background load, not the code. The database was checked for bloat and stray connections first: tables in kilobytes, zero dead tuples after vacuum, zero other connections, zero advisory locks held.	node --env-file=.env.test scripts/load-check.mjs 20 200 20

Coverage by area, measured over the code that ships

Area	Files	Lines	Branches	Functions
Server core (lib/)	16	70%	62%	65%

Area	Files	Lines	Branches	Functions
API handlers (lib/api/)	9	66%	63%	52%
Screens (components/qms/)	17	9%	14%	7%

The screens number is honest and low: two of seventeen components have tests. The server, where the money and the data are, is covered where it matters. The thresholds hold the current line so a change that removes coverage fails; raising the screens number is the next testing task and is listed in section 5.

4. Decisions kept on purpose, and why

A reviewer who disagrees with any of these should find the argument here, not have to reconstruct it.

Decision	What a stricter reading wants	Why it stands
The application role may call every function, including retention and the routing tick	Reserve maintenance functions for the owner or a third role	The scheduler runs as the same role by design, and it is the process that needs those two functions. A third credential for two functions adds a secret to rotate and a failure mode for a gain that only matters if the web role is compromised, in which case the attacker already holds every ticket. Revisit if the scheduler ever gets its own role.
Three stylesheets remain	One styling system	Measured: 202 of 623 selectors are defined in more than one file. The 91 blocks that were provably dead are gone. The remaining 230 overlaps change what the customer sees if touched, so they are a design decision to be made with a designer and a screen, not a code change to be made blind before a demonstration.
Validation does not key on NODE_ENV	Refuse an http origin in production	The first version did exactly that, and refused to start on a laptop, because the production build is the same artefact everywhere. The rule that actually protects users is that the cookie's Secure flag agrees with the origin's scheme; that is what is enforced. An http origin is allowed and logged as insecure at boot, because policing where a build runs is an operator's decision, not the code's.
Business rules in PostgreSQL, one advisory lock, the API inside Next.js, forward-only migrations	See the response of 22 September to the first review	Unchanged and unchallenged by this review. The lock was measured then (96 to 214 ms per tick with 200 to 400 waiting) and measured again after the indexes; see section 3.
Substring search and offset paging on the queue	Trigram index and keyset paging	The queue holds today's tickets, a few dozen rows. Both techniques are ready to hand and become worthwhile when the history is searched, which no screen does yet.

5. What remains

Item	Type	Owner
Component tests for the remaining screens (team, queues, reports, settings, TV, audit, ticket drawer)	Engineering, not blocking	Developer
One styling system, with a designer	Design decision	Product, Developer
Metrics and tracing beyond the request log and the alert endpoint	When there is more than one centre	Developer
Routing policy, retention periods, identity provider, production hosting, Salesforce production org, device acceptance	Unchanged from the first response	Business, IT, Salesforce team

6. How the next reviewer verifies this

```
git clone https://github.com/pruthviiii/Samana-QMS.git && cd Samana-QMS && npm ci
# any PostgreSQL 14+: create samana_qms_test, put its URL in .env.test with
# APP_ORIGIN=http://qms.test and SESSION_COOKIE_SECURE=false
node --env-file=.env.test scripts/migrate.mjs
npm run typecheck && npm run lint && npm run test:coverage
npm run build && npm run build:worker
# policy proof: start the built server, fetch /, and confirm every script tag
# carries the nonce named in the Content-Security-Policy header
# load proof: node --env-file=.env.test scripts/load-check.mjs 20 200 20
```

CI runs the same checks on every push: types, lint, the unit and browser projects, the dependency audit, both builds, and the full suite with coverage thresholds against a PostgreSQL 18 container, with no external secret.

7. Change log

Commit	Date	Scope
7486861	22 Sep	Response to the first review, with the defence section
78705c2	22 Sep	This review: configuration boundary and boot validation; nonce script policy with per-request rendering; migration 016 (four indexes, atomic limiter); scheduler on two cadences with shutdown; typed errors with stable codes and 500 semantics; Salesforce probe cache and breaker narrowing; role groups shared by both sides; framework router; refs in effects; dead rules removed; display session kept; browser test project, configuration and policy suites, lifecycle scenario isolated, coverage thresholds in CI; 91 dead stylesheet blocks removed with proof; timers extracted from the shell

Companion documents: `docs/review-response-2026-09-22.html` (the response to the first review), `docs/review-checklist-2026-09-21.md` section I, `docs/requirements.md` decisions, `.env.example`, and the design document "Samana QMS – Demo Flow and Technical Design".